beelogger STM32 LoRa: OTAA in TTN einrichten

Firmware STM32_LORA_261107 · BluePill F103 und BlackPill F401/F411 · Stand 08.11.2026

1. Was bei OTAA anders ist

Bei ABP trägt man Gerätadresse und zwei Sitzungsschlüssel fest in den Sketch ein. Bei OTAA trägt man nur **DevEUI**, **JoinEUI** und **AppKey** ein. Beim ersten Senden meldet sich der Logger selbst im Netz an („Join“) und bekommt seine Gerätadresse und Schlüssel vom Netz.

- Application, Payload-Formatter und Webhook zum beelogger-Server bleiben **genau wie bei ABP**. Neu ist nur das Anlegen des Geräts.
- Der Logger merkt sich die Anmeldung (F4: Backup-Register, F103: letzte Flash-Seite). Nach einem normalen Neustart meldet er sich nicht neu an, der Zähler läuft weiter.
- Neu anmelden tut er sich nur nach Stromausfall (Akku ab), wenn er etwa einmal am Tag dreimal hintereinander keine Antwort vom Netz bekommt, und kurz bevor der Zähler 16 000 erreicht.
- Der Join dauert beim ersten Senden einige Sekunden länger, danach ist ein Zyklus so kurz wie bei ABP.

2. Gerät in TTN anlegen

TTN-Konsole (`eu1.cloud.thethings.network/console`) → eigene Application → **End devices → Register end device**.

Feld	Wert
Input method	Enter end device specifics manually
Frequency plan	Europe 863-870 MHz (SF9 for RX2 – recommended)
LoRaWAN version	LoRaWAN Specification 1.0.2
Regional Parameters version	RP001 Regional Parameters 1.0.2 **revision B**

Dann auf **Show advanced activation, LoRaWAN class and cluster settings** klicken:

Feld	Wert
Activation mode	Over the air activation (OTAA)
Additional LoRaWAN class capabilities	None (class A only)
Network defaults: „Use network's default MAC settings“	Haken **weg**
Rx2 data rate	**3**
Rx2 frequency	869.525 MHz
Skip registration on Join Server	kein Haken

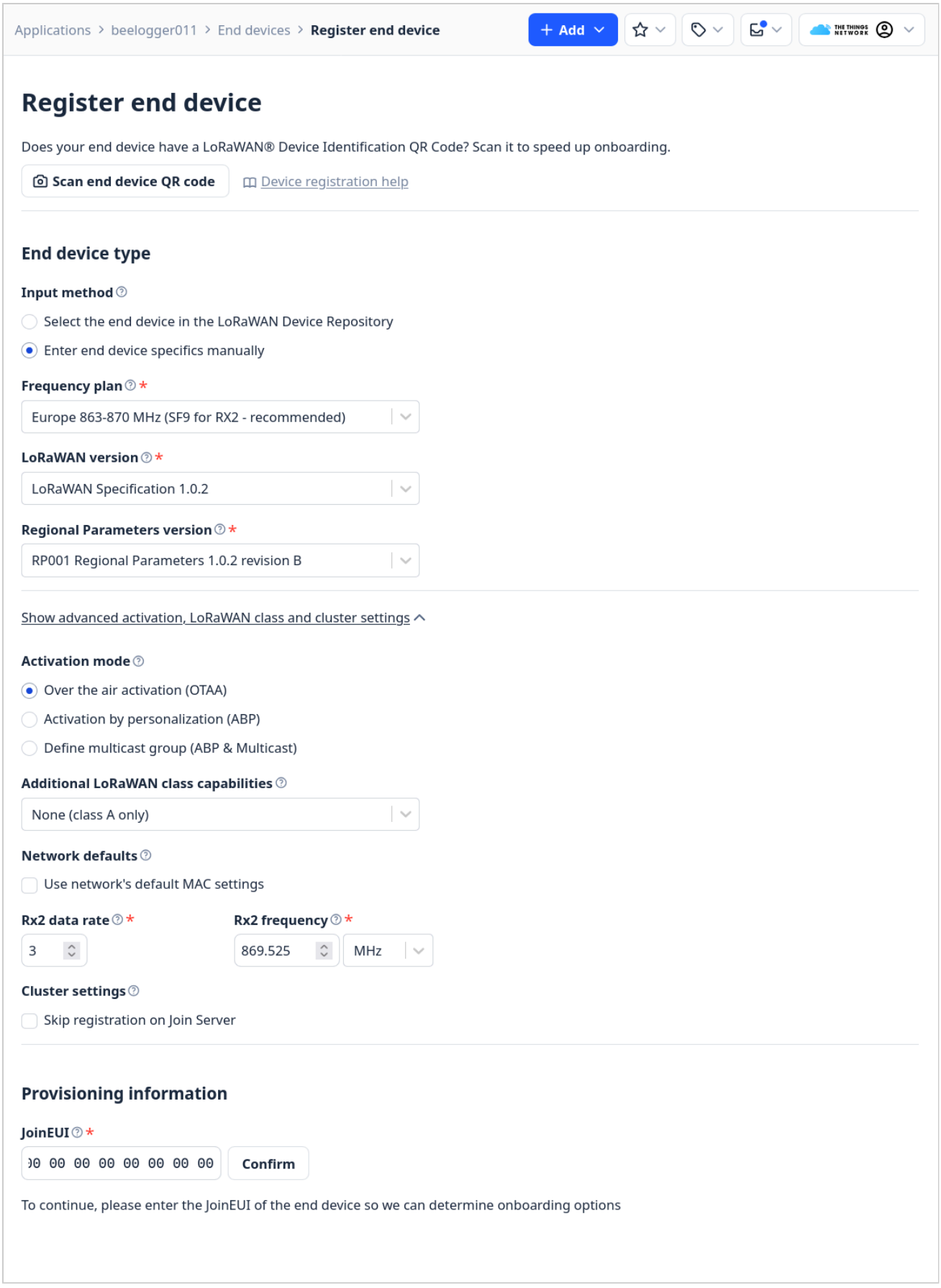


Bild 1: Grunddaten

Unter **Provisioning information**:

1. **JoinEUI**: `00 00 00 00 00 00 00 00` eintragen, **Confirm** klicken.
2. **DevEUI**: **Generate** klicken.
3. **AppKey**: **Generate** klicken.
4. **End device ID**: frei wählbar, z. B. Name des Loggers (nur Kleinbuchstaben, Ziffern, Bindestrich).
5. **Register end device** klicken.

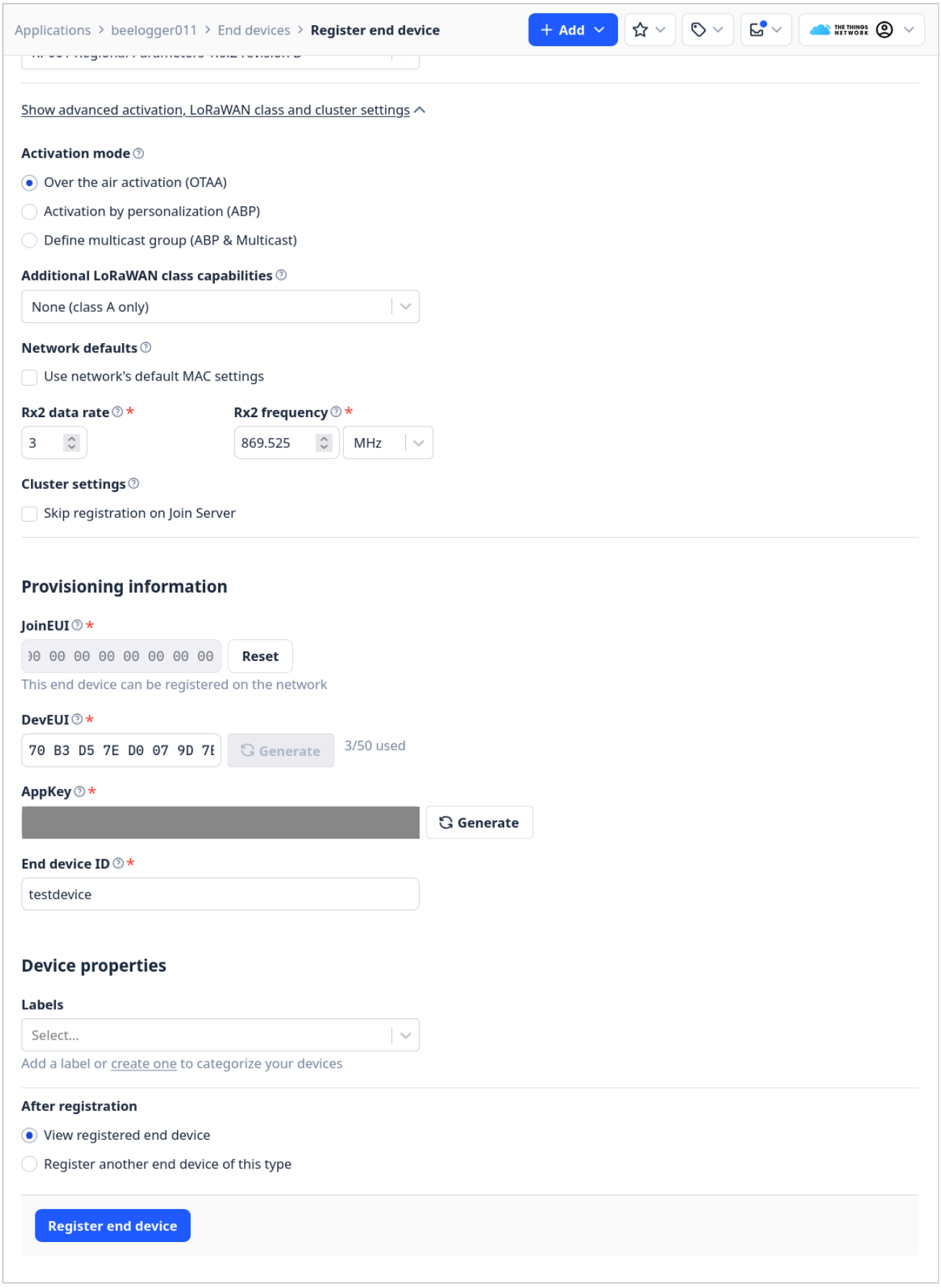


Bild 2: JoinEUI, DevEUI, AppKey (AppKey geschwärzt)

3. MAC-Einstellungen anpassen (wichtig)

Der Logger beantwortet keine Steuerbefehle des Netzes. Stimmen die Einstellungen in TTN nicht, schickt TTN nach jedem Uplink einen Downlink, den niemand braucht (Strom, Fair-Use-Grenze von 10 Downlinks am Tag).

Gerät → **Settings** (General settings) → **Network layer** → Expand → **Custom MAC settings**:

Feld	**Wert**
Frame counter width	32 bit
Desired Rx1 delay	**1** sec (Vorgabe 5)
Desired Rx1 data rate offset	0
Desired Rx2 data rate index	**3**
Desired Rx2 frequency	869,525 MHz
Status count periodicity	**0** (Vorgabe 200)
Status time periodicity	**0** (Vorgabe 86400)
Adaptive data rate (ADR)	**Disabled** (Vorgabe Dynamic mode)

Unten **Save changes**. Die Werte bekommt der Logger beim Join; deshalb vor dem ersten Einschalten einstellen. Wurden sie erst danach geändert: Akku kurz ab, dann meldet er sich neu an.

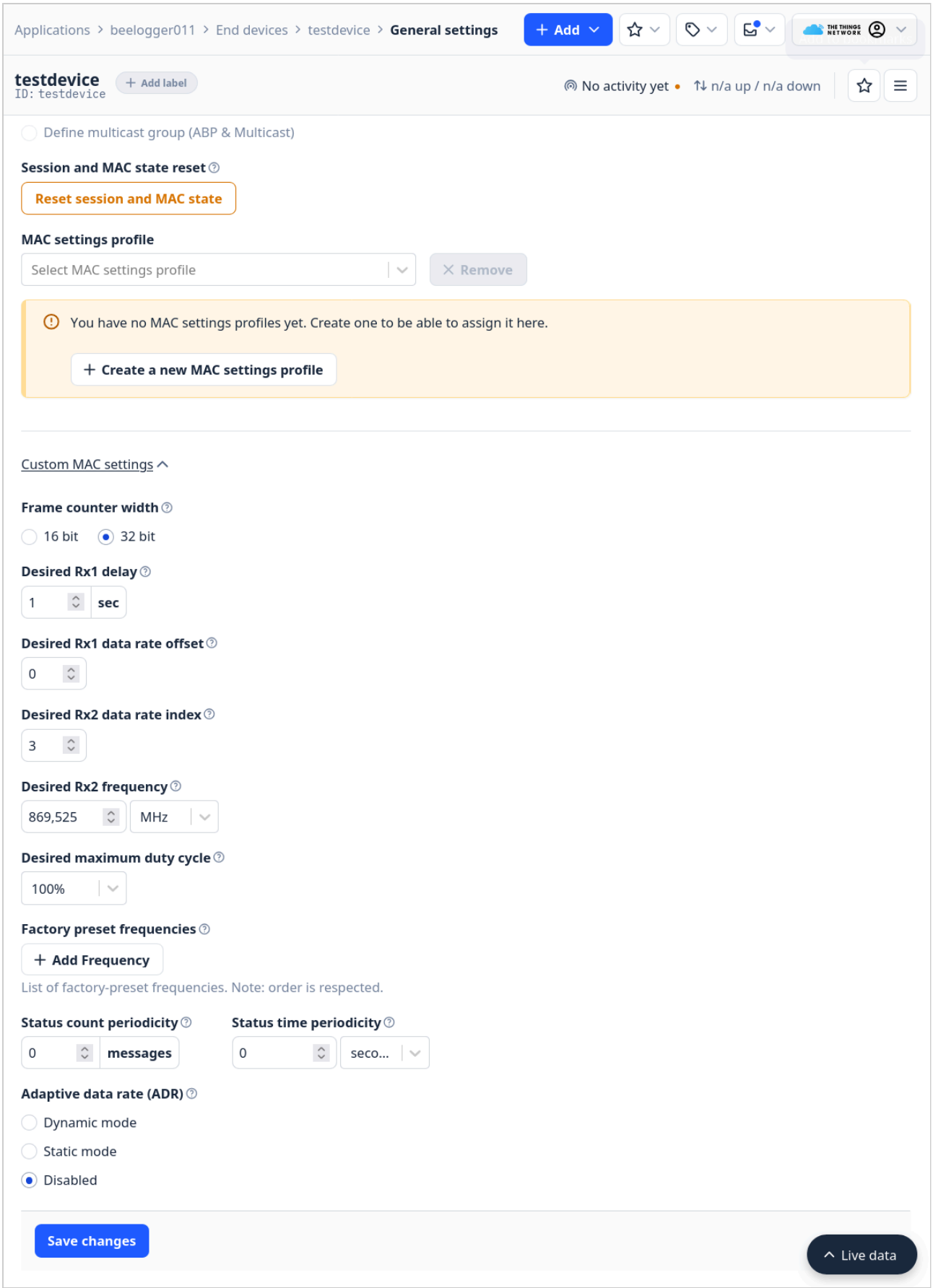


Bild 3: Custom MAC settings

4. Werte in den Sketch eintragen

Datei `Config_LORA.h` im Sketch-Ordner:

1. `#define LORA_OTAA 0` auf `#define LORA_OTAA 1` ändern.
2. `#define LOGGER_NAME "beelogger"`: Namen eintragen (erscheint nur im seriellen Monitor).
3. Die drei Werte aus der TTN-Konsole (Gerät → Device overview) übernehmen. Neben jedem Wert gibt es den Knopf **<>** (Format „C-style“, `{ 0x.., 0x.. }`) und den Knopf **⇄** für die Byte-Reihenfolge (MSB / LSB):

Config_LORA.h	TTN-Feld	Reihenfolge
`DEVEUI[8]`	DevEUI	**LSB**
`APPEUI[8]`	JoinEUI (bei 00… egal)	**LSB**
`APPKEY[16]`	AppKey	**MSB**

```
#define LORA_OTAA 1
#if LORA_OTAA
static const u1_t PROGMEM DEVEUI[8]  = { 0x.., ... };   // LSB
static const u1_t PROGMEM APPEUI[8]  = { 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00
};
static const u1_t PROGMEM APPKEY[16] = { 0x.., ... };   // MSB
```

Häufigster Fehler: falsche Byte-Reihenfolge. Dann schickt der Logger Join-Anfragen, TTN antwortet nicht. Beispiel: Zeigt TTN die DevEUI als `70 B3 D5 7E D0 07 9D 7A`, gehört in den Sketch `{ 0x7A, 0x9D, 0x07, 0xD0, 0x7E, 0xD5, 0xB3, 0x70 }`.

Die ABP-Werte (DEVADDR, NWKSKEY, APPSKEY) bleiben einfach stehen, sie werden bei OTAA nicht benutzt. Die Bibliothek für OTAA liegt im Sketch-Ordner unter `src/lmic_otaa`; die installierte LMIC-Bibliothek bleibt unverändert. Übertragen wie gewohnt mit der Arduino-IDE.

5. Prüfen

1. In TTN beim Gerät **Live data** öffnen, **bevor** der Logger eingeschaltet wird.
2. Logger einschalten. Nach einigen Sekunden erscheinen „Receive join-request“ und „Accept join-request“, danach der erste Uplink mit den Messwerten.
3. Im seriellen Monitor (9600 Baud) steht `OTAA: Join`, `Joining`, `Joined DevAddr: …`, dann `TX done!`.
4. Beim nächsten Zyklus kein Join mehr, nur Uplink.

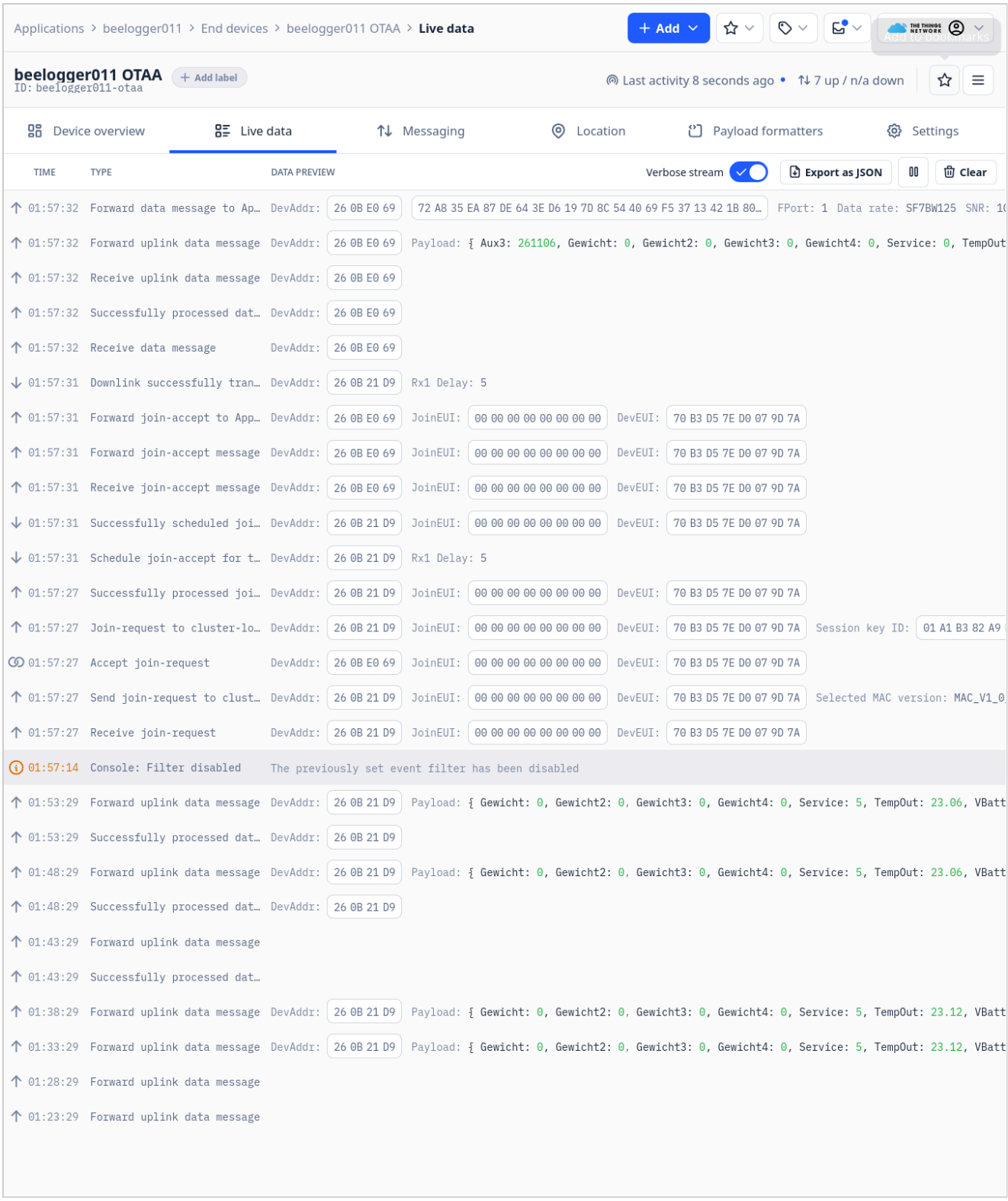


Bild 4: Live data, von unten nach oben: Uplinks, Join, erster Uplink nach dem Join

6. Wenn es nicht geht

Was man sieht	Ursache	Abhilfe
Kein „Joined“ nach 30 s, in TTN keine oder unbeantwortete Join-Requests	DevEUI/JoinEUI nicht LSB oder AppKey nicht MSB	Reihenfolge prüfen (Abschnitt 4), neu übertragen

In TTN gar nichts	kein Gateway in Reichweite, Antenne	TTN-Mapper, Standort prüfen
Nach jedem Uplink ein Downlink, den niemand geschickt hat	MAC-Einstellungen (Abschnitt 3) fehlen oder falsch	korrigieren, Save changes, Akku kurz ab (Neu-Join)
Join klappt, Werte kommen nicht auf dem beelogger-Server an	Webhook oder Formatter der Application	wie bei ABP prüfen

Hinweis Datenmenge (TTN Fair Use): höchstens 30 s Sendezeit am Tag je Gerät. Bei 5 min Intervall liegt ein Logger mit vier Waagen bei etwa drei Vierteln davon; 10 min sind entspannt.